

Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 7

1 - Enunciado

Implemente um programa em C que cria uma lista encadeada com nós, cada um, contendo o nome de um arquivo fictício. Dentro de uma região paralela, percorra a lista e crie uma tarefa com `#pragma omp task` para processar cada nó. Cada tarefa deve imprimir o nome do arquivo e o identificador da thread que a executou com o `omp_get_thread_num`. Após executar o programa, reflita: todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado? O comportamento muda entre execuções? Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

2 - Metodologia

Com base na explicação do enunciado foram desenvolvidos dois códigos um sequencial e outro paralelo, o sequencial para fim de teste e o paralelo para aplicar os conceitos requisitados no enunciado, a compilação dos códigos foi feita da seguinte forma:

```
gcc -fopenmp list_sequence.c -o list_sequence
time ./list_sequence

gcc -fopenmp list_parallel.c -o list_parallel
export OMP_NUM_THREADS=12; time ./list_parallel
```

2.3 - Ambiente de Execução

Componente	Especificação
Notebook	Lenovo IdeaPad 3 15ALC6
SO	Ubuntu 26.04 LTS
Kernel	Linux 7.0.0-29-generic
CPU	AMD Ryzen 5 5500U — 6C/12T
Arquitetura	x86_64
RAM	9,05 GiB
Desktop	GNOME 50.1
Shell	Zsh 5.9
Terminal	Kitty 0.45.0
Compilador	GCC

3 - Código Fonte

3.1 código sequencial

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <string.h>

#define TAM_NOME 100

typedef struct No{
    char nome[TAM_NOME];
```

```

    struct No *prox;
} No;

No *init(const char *nome){
    No *novoNo = (No *) malloc(sizeof(No));
    if(novoNo == NULL){
        printf("Erro!");
        return NULL;
    }

    strncpy(novoNo->nome, nome, TAM_NOME - 1);
    novoNo -> nome[TAM_NOME - 1] = '\0';
    novoNo -> prox = NULL;
    return novoNo;
}

void Add(No **cabeca, const char *nome){
    No *novoNo = init(nome);
    if(*cabeca == NULL){
        *cabeca = novoNo;
        return;
    }

    No *noAtual = *cabeca;

    while(noAtual->prox != NULL){
        noAtual = noAtual->prox;
    }
    noAtual->prox = novoNo;
}

```

```

int verificar(const No *cabeca){
    int count = 0;
    const No *atual = cabeca;
    while(atual != NULL){
        count++;
        atual = atual->prox;
    }
    return count;
}

void liberar(No *cabeca){
    No *noAtual = cabeca;

    while(noAtual != NULL){
        No *proximo = noAtual->prox;
        free(noAtual);
        noAtual=proximo;
    }
}

void processar(const No *no){
    printf("Nome: %25s | thread: %d\n", no->nome, omp_get_thread_num());
}

int main(){
    No *lista = NULL;
    const char *filmes[] = {
        "star_wars_uma_nova_esperanca.mp4",
        "o_senhor_dos_aneis_a_sociedade_do_anel.mp4",
    }
}

```

```
"matrix.mp4",
"blade_runner_2049.mp4",
"duna.mp4",
"interestelar.mp4",
"de_volta_para_o_futuro.mp4",
"jurassic_park.mp4",
"alien_o_oitavo_passageiro.mp4",
"o_exterminador_do_futuro.mp4",
"vingadores_ultimato.mp4",
"guardioes_da_galaxia.mp4",
"homem_aranha_no_aranhaverso.mp4",
"jogador_numero_1.mp4",
"tron_o_legado.mp4",
"ghost_in_the_shell.mp4",
"akira.mp4",
"circulo_de_fogo.mp4",
"no_limite_do_amanha.mp4",
"millennium_falcon_edicao_especial.mp4"
};

int n_filmes = sizeof(filmes)/sizeof(filmes[0]);

for(int i = 0; i < n_filmes; i++){
    Add(&lista, filmes[i]);
}

int total_nos = verificar(lista);
printf("Total de nos na Lista: %d \n\n", total_nos);

No *atual = lista;
```

```

while(atual != NULL){
    processar(atual);
    atual = atual->prox;
}

liberar(lista);
return 0;
}

```

3.2 - list_parallel.c

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <string.h>

#define TAM_NOME 100

typedef struct No{
    char nome[TAM_NOME];
    struct No *prox;
} No;

No *init(const char *nome){
    No *novoNo = (No *) malloc(sizeof(No));
    if(novoNo == NULL){
        printf("Erro!");
        return NULL;
    }
}

```

```
    strncpy(novoNo->nome, nome, TAM_NOME - 1);  
    novoNo -> nome[TAM_NOME - 1] = '\0';  
    novoNo -> prox = NULL;  
    return novoNo;  
}
```

```
void Add(No **cabeca, const char *nome){  
    No *novoNo = init(nome);  
    if(*cabeca == NULL){  
        *cabeca = novoNo;  
        return;  
    }  
}
```

```
    No *noAtual = *cabeca;  
  
    while(noAtual->prox != NULL){  
        noAtual = noAtual->prox;  
    }  
    noAtual->prox = novoNo;  
}
```

```
int verificar(const No *cabeca){  
    int count = 0;  
    const No *atual = cabeca;  
    while(atual != NULL){  
        count++;  
        atual = atual->prox;  
    }  
    return count;  
}
```

```

}

void liberar(No *cabeca){
    No *noAtual = cabeca;

    while(noAtual != NULL){
        No *proximo = noAtual->prox;
        free(noAtual);
        noAtual=proximo;
    }
}

void processar(const No *no){
    printf("Nome: %25s | thread: %d\n", no->nome, omp_get_thread_num());
}

int main(){
    No *lista = NULL;
    const char *filmes[] = {
        "1- star_wars_uma_nova_esperanca.mp4",
        "2- o_senhor_dos_aneis_a_sociedade_do_anel.mp4",
        "3- matrix.mp4",
        "4- blade_runner_2049.mp4",
        "5- duna.mp4",
        "6- interestelar.mp4",
        "7- de_volta_para_o_futuro.mp4",
        "8- jurassic_park.mp4",
        "9- alien_o_oitavo_passageiro.mp4",
        "10- o_exterminador_do_futuro.mp4",
        "11- vingadores_ultimato.mp4",
    }
}

```

```
"12- guardioes_da_galaxia.mp4",  
"13- homem_aranha_no_aranhaverso.mp4",  
"14- jogador_numero_1.mp4",  
"15- tron_o_legado.mp4",  
"16- ghost_in_the_shell.mp4",  
"17- akira.mp4",  
"18- circulo_de_fogo.mp4",  
"19- no_limite_do_amanha.mp4",  
"20- millennium_falcon_edicao_especial.mp4"  
};
```

```
int n_filmes = sizeof(filmes)/sizeof(filmes[0]);
```

```
for(int i = 0; i < n_filmes; i++){  
    Add(&lista, filmes[i]);  
}
```

```
int total_nos = verificar(lista);  
printf("Total de nos na Lista: %d \n\n", total_nos);
```

```
#pragma omp parallel  
{  
    #pragma omp single  
    {  
        No *atual = lista;  
        while(atual != NULL){  
            No *no = atual;  
            #pragma omp task firstprivate(no)  
            {  
                processar(no);  
            }  
        }  
    }  
}
```

```

    }

    atual = atual->prox;

}

#pragma omp taskwait

}

}

liberar(lista);

return 0;

}

```

4 - Resultado

O código foi implementado de duas maneiras, a primeira como nas tarefas anteriores foi de forma sequencial e através desse código sequencial foi feita alterações usando as cláusulas requisitadas na tarefa e dessa forma foi obtido os seguintes resultados.

```

> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_7
> time ./list_sequence
Total de nos na Lista: 20

Nome: star_wars_uma_nova_esperanca.mp4 | thread: 0
Nome: o_senhor_dos_aneis_a_sociedade_do_anel.mp4 | thread: 0
Nome: matrix.mp4 | thread: 0
Nome: blade_runner_2049.mp4 | thread: 0
Nome: duna.mp4 | thread: 0
Nome: interestelar.mp4 | thread: 0
Nome: de_volta_para_o_futuro.mp4 | thread: 0
Nome: jurassic_park.mp4 | thread: 0
Nome: alien_o_oitavo_passageiro.mp4 | thread: 0
Nome: o_exterminador_do_futuro.mp4 | thread: 0
Nome: vingadores_ultimato.mp4 | thread: 0
Nome: guardioes_da_galaxia.mp4 | thread: 0
Nome: homem_aranha_no_aranhaverso.mp4 | thread: 0
Nome: jogador_numero_1.mp4 | thread: 0
Nome: tron_o_legado.mp4 | thread: 0
Nome: ghost_in_the_shell.mp4 | thread: 0
Nome: akira.mp4 | thread: 0
Nome: circulo_de_fogo.mp4 | thread: 0
Nome: no_limite_do_amanha.mp4 | thread: 0
Nome: millennium_falcon_edicao_especial.mp4 | thread: 0
./list_sequence 0,00s user 0,00s system 79% cpu 0,005 total

```

figura 1: saída sem a paralelização

Ao comentar o seguinte trecho podemos identificar que as threads estão acessando e alterando o mesmo dado ao mesmo tempo:

```
#pragma omp parallel
{
    // #pragma omp single
    {
        No *atual = lista;

        while(atual != NULL){
            No *no = atual;

            #pragma omp task firstprivate(no)
            {
                processar(no);
            }

            atual = atual->prox;
        }

        #pragma omp taskwait
    }
}
```

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_7
> gcc -fopenmp list_parallel.c -o list_parallel
> export OMP_NUM_THREADS=12; time ./list_parallel
Total de nos na Lista: 20

Nome: 20- millennium_falcon_edicao_especial.mp4 | thread: 8
Nome: 19- no_limite_do_amanha.mp4 | thread: 8
Nome: 18- circulo_de_fogo.mp4 | thread: 8
Nome: 17- akira.mp4 | thread: 8
Nome: 20- millennium_falcon_edicao_especial.mp4 | thread: 9
Nome: 20- millennium_falcon_edicao_especial.mp4 | thread: 0
Nome: 19- no_limite_do_amanha.mp4 | thread: 0
Nome: 18- circulo_de_fogo.mp4 | thread: 0
Nome: 17- akira.mp4 | thread: 0
Nome: 16- ghost_in_the_shell.mp4 | thread: 0
Nome: 20- millennium_falcon_edicao_especial.mp4 | thread: 3
Nome: 19- no_limite_do_amanha.mp4 | thread: 3
Nome: 18- circulo_de_fogo.mp4 | thread: 3
Nome: 17- akira.mp4 | thread: 3
Nome: 16- ghost_in_the_shell.mp4 | thread: 3
Nome: 15- tron_o_legado.mp4 | thread: 3
Nome: 14- jogador_numero_1.mp4 | thread: 3
Nome: 13- homem_aranha_no_aranhaverso.mp4 | thread: 3
Nome: 12- guardioes_da_galaxia.mp4 | thread: 3
Nome: 11- vingadores_ultimato.mp4 | thread: 3
Nome: 10- o_exterminador_do_futuro.mp4 | thread: 3
```

Figura 2 : saída com vários acessos ao nó

Depois de identificar os erros dos nós sendo acessados mais de uma vez por mais de uma thread e uma thread acessando mais de um dado ao mesmo tempo:

```
> felix@felix-IdeaPad-3-15ALC6:~/Documentos/git/Programacao_Paralela/Tarefa_7
> export OMP_NUM_THREADS=12; time ./list_parallel
Total de nos na Lista: 20

Nome: 1- star_wars_uma_nova_esperanca.mp4 | thread: 10
Nome:      3- matrix.mp4 | thread: 11
Nome: 2- o_senhor_dos_aneis_a_sociedade_do_anel.mp4 | thread: 0
Nome: 14- jogador_numero_1.mp4 | thread: 0
Nome:      15- tron_o_legado.mp4 | thread: 0
Nome: 16- ghost_in_the_shell.mp4 | thread: 0
Nome:      17- akira.mp4 | thread: 0
Nome: 18- circulo_de_fogo.mp4 | thread: 0
Nome: 13- homem_aranha_no_aranhaverso.mp4 | thread: 11
Nome:      5- duna.mp4 | thread: 2
Nome: 7- de_volta_para_o_futuro.mp4 | thread: 7
Nome:      8- jurassic_park.mp4 | thread: 4
Nome: 10- o_exterminador_do_futuro.mp4 | thread: 6
Nome: 20- millennium_falcon_edicao_especial.mp4 | thread: 3
Nome: 11- vingadores_ultimato.mp4 | thread: 9
Nome: 4- blade_runner_2049.mp4 | thread: 8
Nome: 12- guardioes_da_galaxia.mp4 | thread: 10
Nome: 19- no_limite_do_amanha.mp4 | thread: 0
Nome:      6- interestelar.mp4 | thread: 1
Nome: 9- alien_o_oitavo_passageiro.mp4 | thread: 5
./list_parallel 0,12s user 0,00s system 660% cpu 0,019 total
```

Figura 3: saída do código usando task corretamente

5 - Conclusão

Esse experimento foi de grande valia pois mostrou algo muito importante dentro da programação paralela que diz respeito a manipulação de thread e execução de uma mesma tarefa em concorrência com outras threads, dito isso todos os nós foram processados pela função processa pode ser visualizado nas figuras no tópico 4.

Nos testes feitos em várias execuções os nós foram executados mais de uma vez pela mesma thread ou por threads diferentes figura 2, esse problema foi corrigido ao utilizar `#pragma omp single` isso permite que qualquer thread execute o bloco de código e logo após adiciona uma barreira implícita de sincronização automática no final, o single faz com que apenas uma thread execute o bloco single, enquanto as demais threads podem executar as tasks criadas por esse bloco..

Outra pergunta respondida nos experimentos é que cada iteração do while cria exatamente uma tarefa para o nó atual. Como no é `firstprivate`, cada tarefa recebe sua própria cópia do ponteiro para o nó correspondente e também que a ordem de impressão pode mudar. As tarefas são executadas pelas threads disponíveis, e o OpenMP não garante a ordem em que elas serão executadas.